Tarn

SDK Guide

Building apps on permanent, encrypted, user-owned data

May 2026

Contents

Tarn Client

App-facing SDK for Tarn — permanent, encrypted, user-owned data on Arweave.

The SDK is schema-first: you declare collections and fields up front, and the client gives you typed CRUD per collection plus typed namespaces for connections, sharing, recovery, account management, and sessions. Apps never touch Arweave txids, content-encryption keys, share-log seq numbers, or HPKE handshakes — those stay inside the SDK.

Ships full TypeScript declarations. Works in browsers and Node.js 18+.

Quick start

```
import { TarnClient, defineSchema, TarnStorage } from 'tarn-client';

const schema = defineSchema({
  appId: 'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId:   'string',
        title:    'string',
        body:     'string?',
        priority: 'integer?',
      },
      shareable: true,
    },
  },
});

const tarn = await TarnClient.create({
  apiBase: 'http://localhost:8787',
  appId:   'your-app',
  schema,
  storage: TarnStorage.memory(),
});

await tarn.register('me@example.com', 'p@ssw0rd', { recoveryAcknowledged: true });

await tarn.notes.create({ noteId: 'n1', title: 'Hello, Tarn' });
console.log(await tarn.notes.list());
```

That's the whole shape. Everything below is detail.

Schema

A schema declares what collections exist, what fields each collection has, and which collections can be shared. `defineSchema()` brands the result so the client only accepts validated schemas, and validates the shape eagerly at module load.

```
import { defineSchema } from 'tarn-client';

export const schema = defineSchema({
  appId:   'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId:    'string',
        title:     'string',
        body:      'string?',
        priority:  { type: 'integer', required: false },
        updatedAt: 'date?',
        pinned:    'boolean?',
        status:    { type: 'string', enum: ['draft', 'active', 'archived'] },
      },
      shareable: true,
    },
    settings: {
      primaryKey: 'key',
      fields: {
        key:   'string',
        value: 'json',
      },
      // shareable defaults to false — share()/listShared() are not available.
    },
  },
});
```

Field types: `string`, `number`, `integer`, `boolean`, `date`, `json`. Append `?` for optional (`'string?'`), or use the long form `{ type, required, default, enum }`.

Reserved names: `cred`, `connection`, `share-log-state`, `share-inbox`, `recovery-factor`, `app-config`, `app-schema`. Declaring a collection with a reserved name throws at `defineSchema()`.

Sharing: only collections with `shareable: true` get `share/shareWithAll/unshare/listShared` methods. The schema is the authoritative answer to "is this thing shareable" — apps can't bypass it.

Validation: `tarn.<collection>.create()` and `update()` validate against the schema synchronously, before any network or crypto work. Missing required fields, wrong types, unknown fields, or enum violations throw `TarnSchemaError`.

Schema versioning and field evolution

Schemas carry a numeric `version`. Bumping it republishes the schema (via `tools/publish-schema.mjs`) under a new `v=` Arweave tag so the recovery client can pick the version that matches each record's `SchemaV` tag.

Three rules cover the common cases:

Change	Backward-compat?	What you do
Add a new optional field	yes	Bump `version`, republish. Older records read with the field absent; new writes include it.
Add a new required field with a default	yes	Bump `version`, republish. The default fills in for older records on read.
Add a new required field without a default	**no**	Older records fail validation on read. Either supply a default, or migrate (see below).
Remove a field	**no**	Older records still carry the field on disk; the strict validator rejects unknown fields on read. Either keep the field declared as deprecated (still accepted, no longer used), or migrate.
Rename a field	**no**	Same as remove + add. Migrate, don't rename in place.
Change a field's type	**no**	Always a breaking change. Migrate.
Tighten an enum (drop a value)	**no**	Older records carrying the dropped value fail. Don't drop; deprecate.
Loosen an enum (add a value)	yes	Older records still satisfy the union.

Migration pattern. Tarn doesn't ship a `migrate()` helper — apps write their own walk because the right semantics (one-shot vs. lazy, error handling, partial-failure recovery) are app-specific. The pattern is straightforward:

```
// One-shot migration: read all, transform, re-write under the new schema.
const all = await tarn.notes.list();
for (const note of all) {
  if (note.body == null) {
    await tarn.notes.update(note.noteId, { body: '' });
  }
}
```

Validation runs on `update()`, so the migration loop fails fast if the transform is wrong. Re-running is safe — `update()` is idempotent on identical inputs.

Forward-looking — relaxed validation. The current validator is strict by design: unknown fields throw. A future option (`onUnknownField: 'strip' | 'preserve' | 'error'`) will let apps opt into laxer behaviour for upgrade paths — read with extra fields, log a warning, drop them on the next write. Default stays `'error'` so the schema-first commitment isn't watered down. Not shipped yet; track the issue if it matters for your migration plan.

Collections

Each `shareable: true` collection on the schema becomes a typed namespace on the client.

```
// Create. Validates against the schema. Returns the validated record.
await tarn.notes.create({ noteId: 'n1', title: 'Quarterly review', body: 'Pull metrics'
    });

// Get one. Returns null if no record matches.
const note = await tarn.notes.get('n1');

// List all live records.
const all = await tarn.notes.list();

// Partial update — SDK reads current, merges patch, writes a new entry.
await tarn.notes.update('n1', { priority: 5 });

// Tombstone.
await tarn.notes.delete('n1');
```

Records are addressed by **primary key**, never by Arweave txid. The SDK maps primary keys onto the protocol's `Eid` tag so reads converge across devices.

`update()` is **partial-merge**. Pass only what's changing; the SDK reads the current record from Arweave, merges, re-validates as a full record, and writes a chained entry. Full-replace is `update(id, { ...current, ...patch })` if you ever want it.

Sharing (when `shareable: true`)

```
const friends = await tarn.connections.list();

// Share one record with one friend.
await tarn.notes.share(friends[0], 'n1');

// Share with everyone (skips muted connections). Returns counts and per-conn failures.
const result = await tarn.notes.shareWithAll('n1');
// { ok: 3, failed: [{ connection, error }] }

// Revoke from one friend.
await tarn.notes.unshare(friends[0], 'n1');

// Read what a friend has shared with us under THIS collection.
const theirs = await tarn.notes.listShared(friends[0]);
```

Calling `share()` on a non-`shareable` collection throws — the schema is the gate.

Connections

`tarn.connections.*` covers the full lifecycle of friend connections.

```
// Email-based handshake (requires recipient to be a discoverable Tarn user).
await tarn.connections.invite('alice@example.com');

// Recipient lists incoming requests via the advanced surface, then accepts:
await tarn.connections.accept(requestNonce, { label: 'Alice — work team' });

// Listing.
const conns = await tarn.connections.list();
// [{ share_pub, signing_pub, label?, muted? }, ...]

// Mute / unmute. Mute is per-side, invisible to the other party.
await tarn.connections.mute(conns[0]);
await tarn.connections.unmute(conns[0]);
const muted = await tarn.connections.isMuted(conns[0]);

// Set or change a label after the fact.
await tarn.connections.setLabel(conns[0], 'Alice');

// Remove the connection. They stay around in your share-log history but
// new shares stop flowing.
await tarn.connections.remove(conns[0]);
```

Invite tokens — link / QR flow

For consumer apps where the inviter doesn't know the recipient's email (or the recipient isn't a Tarn user yet):

```
// Inviter — generate a single-use, time-limited link.
const { token_id, invite_url, expires_at } = await tarn.connections.createInvite({
  display_name: 'Maya',
```

```
  expiry_days:  7,    // default 7, server max 30
});
// invite_url is something like:
//   https://app.example.com/invite/<token_id>#<base64url payload_key>
// Share it via QR / messenger / email / etc.

// Recipient — extract token_id from path, payload_key from URL fragment.
const tokenId    = new URL(location.href).pathname.split('/').pop();
const payloadKey = location.hash.slice(1);
await tarn.connections.redeemInvite(tokenId, payloadKey);
// The inviter's SDK auto-accepts the resulting connection request on next poll.
```

The URL fragment (`#`) is never transmitted to the API — the payload key stays on the recipient's device. Tarn stores opaque ciphertext keyed on `token_id` and never sees the inviter's identity.

Recovery

Tarn issues every account a 24-word BIP39 recovery phrase at signup. The phrase is a parallel access path to the user's data — independent of the password. Apps **must** surface the phrase to the user during registration; passing `recoveryAcknowledged: true` is the SDK's way of forcing the conversation.

```
const reg = await tarn.register('me@example.com', 'p@ssw0rd', {
  recoveryAcknowledged: true,
  appName: 'My App',              // PDF branding
});
// reg.recoveryPhrase   — 24-word string
// reg.pdfBytes         — Uint8Array of the rendered PDF
// Hand both to the user immediately. Do NOT persist either.

// Re-render the kit later for the same phrase. Pure client-side; no auth
// required. The caller must supply the phrase — Tarn never persists it.
const pdfBytes = await tarn.recovery.export({ format: 'pdf', phrase });
// or: structured JSON for apps rendering their own format.
const json = await tarn.recovery.export({ format: 'json', phrase });
// json: { phrase, appName, generatedAt }
```

Delivery is the app's job. Tarn renders the kit entirely on the client and never sees the phrase or the PDF — there is no Tarn endpoint that handles plaintext recovery material, even ephemerally. Apps decide how to surface the bytes: a download is the recommended default (universally available, no third party); print and app-operated email are also fine. If the application wants to email the kit, it must operate the forwarder itself — Tarn will not host one, since routing recovery material through Tarn-operated infrastructure would weaken the zero-knowledge guarantee that applies to everything else in the protocol.

Recovery itself goes through the top-level `tarn.recoverAccount()` (auth lifecycle):

```
await tarn.recoverAccount({
  phrase:      '24 words ...',
  newEmail:    'me@example.com',
```

```
  newPassword: 'fresh-password',
});
// All pre-recovery data is decryptable under the new credentials.
```

Account + Session

```
// Rotate credentials (routine email/password change). Existing data stays decryptable.
// Pass phrase to extend the recovery factor to the new generation.
await tarn.account.changeCredentials('new@example.com', 'new-password', { phrase });

// Permanently delete. Tombstones credentials, clears server-side state, wipes local
    session.
await tarn.account.delete();

// Local session.
tarn.session.isLoggedIn();          // boolean — whether keys are loaded
await tarn.session.clear();         // forget local session; user must re-auth

// Server-side session management (one row per device).
const devices = await tarn.session.listDevices();
// [{ sid, createdAt, lastSeenAt, deviceLabel, viaRecovery, isCurrent }, ...]

await tarn.session.revokeDevice(devices[1].sid);   // log one device out
await tarn.session.revokeAllOthers();              // "log out everywhere else"
await tarn.session.revokeAll();                    // log out everywhere including here
```

Apps can attach a device label at login time — `tarn.login(email, password, { deviceLabel: 'Chrome on MacBook' })` — which surfaces in `listDevices()`.

Advanced (escape hatches)

Power-user surface for prototyping, debugging, and use cases the typed namespaces don't cover. Most apps never need this.

```
// Schema-less entry CRUD — bypasses the collection layer entirely.
await tarn.advanced.entries.create('arbitrary-type', { foo: 'bar' });
await tarn.advanced.entries.update(priorTxid, 'arbitrary-type', { foo: 'baz' });
await tarn.advanced.entries.delete(targetTxid, 'arbitrary-type');

// Raw blob fetch + decrypt by shareKey. Useful for one-off recipient flows.
const blob = await tarn.advanced.entries.fetchBlob(txid);
const data = await tarn.advanced.entries.decryptSharedBlob(blob, shareKey);

// Direct share-log access. Collection.share()/listShared() are the typed wrappers.
await tarn.advanced.shareLog.read(connection);
await tarn.advanced.shareLog.share(connection, contentId, txid, shareKey);
await tarn.advanced.shareLog.unshare(connection, contentId);
```

If you find yourself reaching for `advanced.*` for something the typed surface should cover, that's a signal to file an issue.

App registration

Before users can register on your app, the app itself must be registered with Tarn. This is operator-level setup; it happens once per app, not per user.

1. Generate an app key pair

```
node tools/generate-app-key.mjs your-app-id
```

Prints the private key (hex), the public key, and a D1 seed SQL line. **Save the private key in a password manager** — it's printed once.

2. Register the app in Tarn

Run the printed D1 seed command:

```
# Local
cd api && npx wrangler d1 execute tarn-api --local --command "<printed SQL>"
# Production
cd api && npx wrangler d1 execute tarn-api --remote --command "<printed SQL>"
```

3. Set per-account rules

After a user registers, set their write rules:

```
node tools/set-rules.mjs \
  --api  https://api.tarn.dev \
  --app  your-app-id \
  --key  <private key hex> \
  --dlk  <user's data_lookup_key> \
  --plan free        # or annual, clear, deny, or --rules '<JSON>'
```

Plan / type	Effect
`free`	5 entries, 100KB max per entry
`annual`	1000 entries, expires in 1 year
`clear`	Empty rules (allow all)
`deny`	Deny all writes
`max_entries`	`{ limit, since?, app?, entry_type? }`
`max_bytes`	`{ limit }` per entry
`expires`	`{ at }` ISO 8601

Rules are AND logic; unknown rule types fail closed.

4. Publish the schema

The schema is published to Arweave so the future "always access your data" recovery client can read it without going through the Tarn API. Run this once per release that bumps `schema.version`:

```
node tools/publish-schema.mjs \
  --api    https://api.tarn.dev \
  --app    your-app-id \
  --key    <private key hex> \
  --schema ./schema.mjs
```

`--schema` accepts either a `.json` file (serialized schema) or a `.mjs` file with a default export equal to the schema. Re-running with the same `(app, version, schema)` is a safe no-op.

Examples

Four progressive examples live in `../examples/`:

- `01-hello-world` — register, write one record, list it.
- `02-crud` — full CRUD on two collections.
- `03-sharing` — invite-token handshake + share-with-all flow.
- `04-recovery` — register, export PDF, simulate password loss + recoverAccount.

Each example is a standalone npm package linking to this client via `file:../../client`. See `examples/README.md` for setup.

TypeScript

The SDK is written in TypeScript and ships full `.d.ts` declarations. Schema-derived types flow through the client:

```
const tarn = await TarnClient.create({ schema: mySchema, /* ... */ });

await tarn.notes.create({
  noteId: 'n1',
  title:  'Foo',
  // titel: 'typo'    // ✗ TS error: unknown field
});

const note = await tarn.notes.get('n1');
//    ^? { noteId: string; title: string; body?: string; priority?: number; ... }
```

Plain JavaScript works too — the inference simply doesn't run. The runtime validators still enforce the schema at write time.

Security model

- **Client-side encryption.** All data is AES-256-GCM encrypted before leaving the client. The server never sees plaintext.
- **Argon2id KDF.** Memory-hard (m=64 MiB, t=3, p=1) for password→master-key. Sub-keys derived via HKDF-Expand.
- **ECDSA P-256 auth.** Challenge-response signing. No passwords transmitted; server stores only the public key.
- **Per-content CEK + forward-secret DEK rotation.** Each blob is encrypted with its own random CEK, wrapped under a generation-indexed DEK chain. Credential changes append a fresh DEK so post-rotation writes are unreachable from old credentials.
- **Multi-factor DEK chain.** Each chain entry is wrapped twice — once under a password-derived KEK, once under a phrase-derived KEK. Either factor independently unwraps the chain.
- **Arweave permanence.** Data is stored permanently on Arweave. Encrypted blobs are publicly visible but unreadable without the key.

Publicly observable metadata

Tarn protects content end-to-end, but a few metadata properties remain visible:

- **Connection-request inbox volume + timing** is observable to anyone who knows a user's `share_pub`. Casual social use is fine; sensitive contexts should consider `share_discoverable: false`.
- **Account existence via discoverability lookup** — a `share_discoverable: true` account leaks "this email is a Tarn user" to anyone who runs the lookup.
- **Once connected, share-log traffic is unlinkable** — per-pair tags are stealth-addressed; an Arweave observer cannot extract the connection graph from the protocol alone.

See TARN_PROTOCOL.md § Publicly observable metadata for the full breakdown, and TARN_PROTOCOL.md for the wire-protocol spec.